

File System

HDD Secondary Storage provide store and store data permanently

Disc take bola hoy platter

Sector Physical data storage of HDD

Performance metric : - 1) Transfer Rate 2) Positioning Time / Random Access Time

The transfer rate is the rate at which data flow between the drive and the computer. Another performance aspect, the positioning time, or random-access time, consists of two parts: the time necessary to move the disk arm to the desired cylinder, called the seek time, and the time necessary for the desired sector to rotate to the disk head, called the rotational latency

Right Track – Seek Time

Right sector – Rotational Latency

User Data unit —> determines user data in form of files gets stored and can be fetched

Logical Block is the smallest addressable unit in Storage

Disk Controller Firmware maps logical block with physical address

A file is a named collection of related information that is recorded on secondary storage. From a user's perspective, a file is the smallest allotment of logical secondary storage; that is, data cannot be written to secondary storage unless they are within a file.

Truncation→ Remove information but keep attributes (except file length—> file space release korar jonno)

All the file operations except create and delete require a file open ()
jeikhane F.D used as a argument and returns a pointer to the entry in
OpenFileTable
open(r,d,w)

- **Repositioning:** Changes the position of the pointer within a file during IO operations.

write() a ekta write pointer thakbe appending er jonno

Why do we use CFPP ?

—> because a position is either reading from or writing to a file

Seek () ----> moves the file pointer to a specific location

CFPP ke many ways a reposition korte parbo which is known as file seek .This is not an I/O event

The open() operation takes a file name and searches the directory, copying the directory entry into the open-file table. The open() call can also accept access mode information—create, read-only, read-write, append-only, and so on. This mode is checked against the file's permissions. If the request mode is allowed, the file is opened for the process. The open() system call typically returns a pointer to the entry in the open-file table. This pointer, not the actual file name, is used in all I/O operations, avoiding any further searching and simplifying the system-call interface

Each entry in the per process open table points to a system wide open Table .

Mechanisms used for storing a logical block into its corresponding sector by the file system are known as allocation methods.

Contiguous fast because no head movement or seeking and track change
short minimal distance

4. Data Blocks

This is the largest region of the file system where the actual content is stored.

3. Inodes (Inode Table)

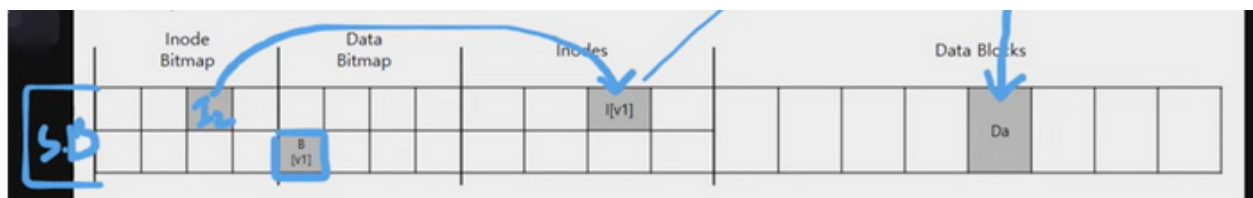
This region stores the actual Inode structures.

2. Data Bitmap

This is similar to the Inode Bitmap but tracks the usage of the disk's data blocks.

1. Inode Bitmap

This is a data structure used to track the availability of inodes. It consists of a sequence of bits where each bit corresponds to a specific inode in the inode table.



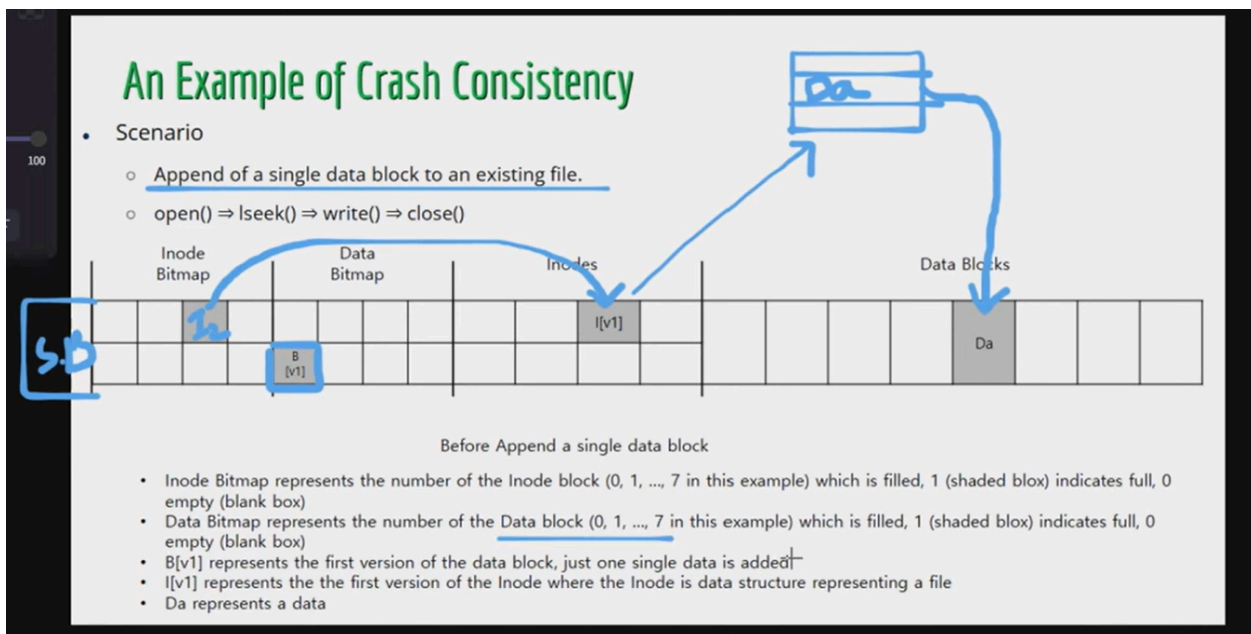
The PDF explains that allocation is tracked using **bitmaps**, where each bit indicates:

- 0 = free
- 1 = in-use

So:

- **Inode bitmap:** tracks which **inodes** are free or allocated (i.e., whether a new file/directory can get an inode).
- **Data bitmap:** tracks which **data blocks** are free or allocated (i.e., where file contents can be placed).

And the superblock's purpose is to store key filesystem metadata used during mount (like number of inodes, inode table start, etc.), so the OS can find these structures.



2. What fields of the file system are updated when we delete a file from the file system ? 2

Inode and data bitmaps

d) Dipu removed a file from the disk which causes the deletion of the last remaining hard link to a file.

[1*2
=2]

- I. **Detect** which two types of resources are now marked as free by the file system?
- II. **State** which fields of the file system are updated to reflect this change?

I) Two types of resources now marked free

1. **The inode** of the file (its metadata structure)
2. **The data blocks** belonging to the file (its contents)

This matches the VSFS/ext2 layout where files use an inode plus data blocks.

II) Which file-system fields/structures are updated

To reflect freeing those resources, the file system updates:

- **Inode bitmap** → marks that inode as **free**
Operating Systems-5-File System...
- **Data bitmap** → marks the file's data blocks as **free**
Operating Systems-5-File System...

Also, since the link was deleted, the **directory data** (the name → inode# entry) is removed/updated; directories are stored as (name, inode#) pairs in the slides.

File delete korar time a data bitmap a oi jayga ta free kore dilei hocche

FSCK (File System Consistency Check).

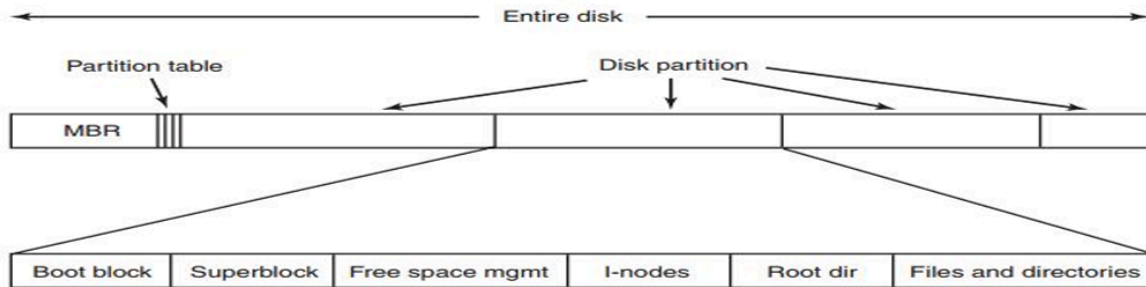


Figure: A possible file-system layout.

- **Truncating:** Removes information from a file. It does not remove attribute of the file.
- **Repositioning:** Changes the position of the pointer within a file during IO operations.

Repositioning not an I/O event

Truncating a all attributes remain unchanged except file length

Linux has extended (ext) file systems etc

File system Allocation —> Non contiguous

- Non-contiguous allocation
 - ☐ Linked allocation
 - ☐ Indexed allocation
 - ☐ UNIX inode

UNIX Inode

- Variant of indexed allocation.
- I means indexed and node means block.
- It maintains multilevel index.

Memory Management

Dynamic Memory (malloc): Sometimes, you don't know how many guests are coming until the day of the party. You need to walk up to the front desk *on the spot* and ask for a room.

Here is the **malloc** process:

1. **You (The Code):** "Hey, I need a room big enough for 5 people (5 integers)."
2. **The Receptionist (malloc):** Checks the hotel ledger (The Heap).
 - **If there is space:** They mark a room as "Occupied" and hand you the **key** (the **pointer**) to that room.
 - **If the hotel is full:** They tell you "Sorry, we are full" (returns **NULL**).

External Fragmentation —> Process sesh hoye geleo majhe empty thakbe

External Fragmentation happens when the *total* free memory is enough to accommodate a process, but the memory is not contiguous (it's split into small pieces), so the process has to wait.

Paging —> Breaking Physical Memory

First Fit

This algorithm scans the memory from the beginning and picks the **very first hole** that is big enough to fit the process. (First Hole that fits)

Best fit

This algorithm searches the **entire** memory list to find the hole that is closest in size to the process. (Smallest hole that fits)

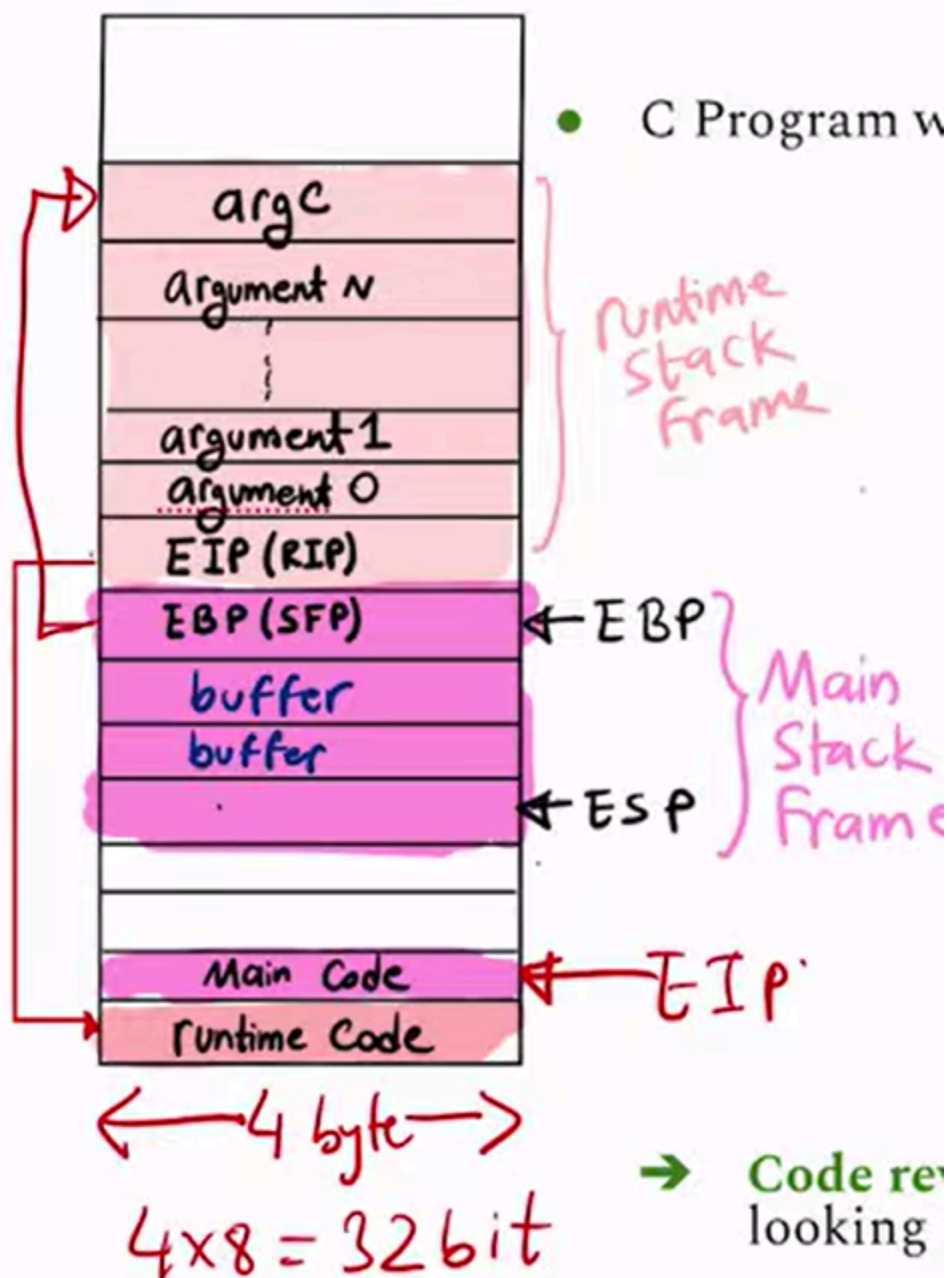
Worst Fit

This algorithm also searches the **entire** list but picks the **largest** available hole. (Biggest hole that fits)

Protection

- ◆ Modern systems implement both MAC and DAC, with MAC usually a more secure, optional configuration (Trusted Solaris, TrustedBSD (used in macOS), SELinux), Windows Vista MAC).

Security



Q/A

File Allocation method contiguous and non contiguous

Memory Management of Contiguous and Non - Contiguous

Summer 25

g) Journaling optimization problem (based on PDF)

I. Why is issuing all five blocks in one request unsafe?

This approach is unsafe because **disks can reorder writes internally**.

Journaling requires **strict write ordering**, where:

- **TxE (transaction end / commit block) must be written last**
- A transaction is considered **committed only if TxE is on disk**

If all blocks (**TxB**, **I[v2]**, **B[v2]**, **Db**, **TxE**) are issued together, the disk may:

- **Persist TxE before the other blocks**

This **violates the write-ahead logging rule**, breaking journaling correctness.

Operating Systems-5-File System...

II. What incorrect data might be replayed during recovery?

During recovery, the filesystem:

- **Scans the log**
- **Replays any transaction that has TxE present**

If a crash occurs after **TxE is written but before other blocks**:

- Recovery assumes the transaction is valid
- It may replay **incorrect or garbage data**, such as:
 - **Incorrect inode (I[v2])**
 - **Incorrect bitmap (B[v2]) → space leak or inconsistency**
 - **Old or unrelated data block (Db)**

This results in **filesystem inconsistency**, even though journaling is used.

Operating Systems-5-File System...

Final takeaway (from PDF)

- **Problem:** Disk write reordering
- **Violation:** Commit written before data
- **Result:** Wrong metadata or data replayed after crash
- **Correct method:**
Write all blocks **except TxE first**, then write **TxE last** as a separate commit write

This is exactly the solution described in the “**To avoid data being inconsistent**” section of the PDF.

h) Dipto is a system administrator. To improve performance, he switches the mechanism of the journaling to metadata journaling in his system.

- I. **Identify** the key difference between data journaling and metadata journaling?
- II. **State** the critical write ordering principle maintained by metadata journaling to ensure the consistency.

I. Key difference: Data journaling vs Metadata journaling

- **Data journaling:** the journal (log) stores **everything in the transaction**, including **metadata + the actual data block(s)** (e.g., `TxB I[v2] B[v2] Db TxE`). [Operating Systems-5-File System...](#)
- **Metadata journaling (ordered journaling):** the journal stores **only metadata** (e.g., `TxB I[v2] B[v2] TxE`). The **data blocks are NOT written into the journal**, which reduces cost because data would otherwise be written twice (log + final place). [Operating Systems-5-File System...](#)

II. Critical write-ordering principle in metadata journaling (to ensure consistency)

Write the data block to its final location *before* committing the metadata in the journal.

In the PDF's ordered/metadata journaling timeline:

1. **Data write** → write data to its final on-disk location
2. **Journal metadata write + commit (TxE)** → log/commit metadata
3. **Checkpoint metadata** → write metadata to home locations later [Operating Systems-5-File System...](#)

This ordering ensures the filesystem never has **committed metadata pointing to data that wasn't safely written yet**. [Operating Systems-5-File System...](#)

i) Kabbya executes an application which deletes a directory, freeing its data block (block #123). Immediately after, it creates a new file which reuses that same block #123 for its own data. A crash occurs before the transaction involving the directory deletion is cleared from the journal.

- I. **State** what issue can arise during the recovery procedure when it replays the old transaction for the directory according to the scenario?
- II. **What** type of record does **ext3** add to the journal to prevent this problem?

I. What issue can arise during recovery replay?

This is the “**block reuse**” problem.

Because the **old directory transaction** (for the deleted directory) is still in the journal, after the crash the recovery process may **replay that old directory block** back to its “final address” (block #123). But block #123 has already been **reused as data for the new file**.

✓ So recovery can **overwrite the new file's data** with the **old directory contents**, causing **data corruption**.

Operating Systems-5-File System...

II. What record does ext3 add to prevent this?

ext3 adds a **revoke record** to the journal.

A **revoke record** tells recovery:

“Do not replay this old metadata block,”
so it won't overwrite the reused block.

Operating Systems-5-File System...

Sources

Write the answers in the answer script.

a) A C++ program uses a pointer that holds the address **0x400**. When the program runs, the hardware actually accesses memory location **0x18400**. [1*2
=2]

- I. **Identify** specifically which two types of addresses are involved in this scenario?
- II. **State** what hardware unit is responsible for this conversion.

I. Two types of addresses involved:

- **Logical (virtual) address:** **0x400** (generated by the CPU/program) Operating Systems-6-Memory Mana...
- **Physical address:** **0x18400** (the actual memory location accessed by hardware)
Operating Systems-6-Memory Mana...

II. Hardware unit responsible for the conversion:

- **Memory Management Unit (MMU)** (does logical → physical address translation)
Operating Systems-6-Memory Mana...

b) A process with a memory requirement of **145698 Bytes** is running on a system. **18 pages** are required to allocate the process in the system.

- I. **Calculate** the page size of the system .
- II. **Calculate** the **internal fragmentation** of the process after paging.

c) A process attempts to access a memory address that belongs to page number **7**. The system's page table has the information of frame numbers of the pages from **0** to **5**, but not for **6** and **7**.

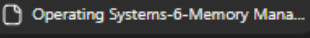
- I. **State** which attribute within the page table is used to identify the illegal page access?
- II. **Write** how the OS uses this mechanism to differentiate between legal and illegal pages for a process?

b)

I. Page size of the system

Process size = 145698 bytes, pages needed = 18

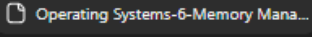
$$\text{Page size} = \left\lceil \frac{145698}{18} \right\rceil = \lceil 8094.33 \dots \rceil = 8095 \text{ bytes}$$

Answer: 8095 bytes/page 

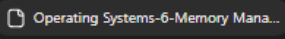
II. Internal fragmentation after paging

$$\text{Allocated space} = 18 \times 8095 = 145710 \text{ bytes}$$

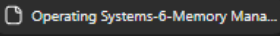
$$\text{Internal fragmentation} = 145710 - 145698 = 12 \text{ bytes}$$

Answer: 12 bytes 

I. Page-table attribute used to identify illegal page access:

Valid-Invalid bit in the page table entry. 

II. How the OS uses it to differentiate legal vs illegal pages:

- Pages that are part of the process's logical address space are marked **valid**.
- Pages not belonging to the process are marked **invalid**.
- If the process references an **invalid** page (like page 7 here), the hardware/OS triggers a **trap (page fault/illegal access)** to the kernel. 

d) In a 32-bit system with a **two-level** page table, the logical address is split into a **9-bit outer page number**, a **9-bit inner page number**, and a **14-bit offset**.

[1*2
=2]

- I. **Calculate** how many entries can the inner page table hold in this system?
- II. **State** what does each entry in the inner page table point to?

g) Consider a system with TLB miss ratio of **4.25%**, and the main memory access time of **125 ns**. If Effective Access Time is **142.3125 ns**, then **calculate** the TLB access time. [1*2
=2]

h) A system is processing the reference string **7, 0, 1, 2, 0, 3, 0, 4,** with three frames, which currently hold pages **{7, 0, 1}**. The next reference is to **page 2**, requiring a replacement. The upcoming references are **....., 0, 3, 0, 4,** [1*2
=2]

- I. If the system uses **Optimal** page replacement algorithm and the CPU demands for **page 2** then find out which page should be swapped out in order to swap in the demanded page in this scenario?
- II. **Justify** your selection based on the future reference strings.

g) Find TLB access time ϵ

Given:

- TLB miss ratio = 4.25% $\Rightarrow$ hit ratio $\alpha = 1 - 0.0425 = 0.9575$
- Main memory access time $m = 125$ ns
- Effective Access Time EAT = 142.3125 ns

From the EAT model (TLB hit: $\epsilon + m$; TLB miss: $\epsilon + 2m$): Operating Systems-6-Memory Mana...

$$EAT = \alpha(\epsilon + m) + (1 - \alpha)(\epsilon + 2m)$$

$$142.3125 = 0.9575(\epsilon + 125) + 0.0425(\epsilon + 250)$$

$$142.3125 = \epsilon + (0.9575 \cdot 125) + (0.0425 \cdot 250)$$

$$0.9575 \cdot 125 = 119.6875, \quad 0.0425 \cdot 250 = 10.625$$

$$142.3125 = \epsilon + 130.3125 \Rightarrow \epsilon = 12 \text{ ns}$$

✓ TLB access time = 12 ns Operating Systems-6-Memory Mana...

h) Optimal page replacement

Frames currently: {7, 0, 1}

Next demanded page: 2 (so one page must be replaced)

Future references: ..., 0, 3, 0, 4, ...

I. Which page should be swapped out (Optimal)?

Swap out page 7 (*page 1 would also be acceptable with the given future string*).  Operating Systems-6-Memory Mana...

II. Justification using future references

Optimal replaces the page **not used for the longest time in the future**.  Operating Systems-6-Memory Mana...

From the upcoming references 0, 3, 0, 4:

- Page 0 is used immediately (very soon) → **do not replace**
- Pages 7 and 1 do **not appear** in the shown future references → their “next use” is farthest/unknown

So replacing 7 (or 1) matches Optimal; choosing 7 is valid because it is **not needed in the given future string**.  Operating Systems-6-Memory Mana...

   ...

- i) A process runs in a system with **3-level** paging and it has a logical address space of **16-bits**. In the system page size is **64 Bytes**, size of each entry of the page table is **4 Bytes** and size of the main memory is **2 KB**. After the paging the logical address space of the process looks like: [2*2=4]

2 nd Outer Page #	Outer Page #	Inner Page #	Offset
P1	P2	P3	d
2	4	4	6

Necessary page table information is given below:

2 nd Outer PMT		
Page #	Frame #	v/i bit
0		0
1	27	1
2	13	1
3		0

Outer PMT						Inner PMT					
Frame 13			Frame 27			Frame 2			Frame 9		
Page #	Frame #	v/i bit	Page #	Frame #	v/i bit	Page #	Frame #	v/i bit	Page #	Frame #	v/i bit
0			0			0			0		
1			1			1	6	1	1	39	1
2			2			2			2		
3	99	1	3			3			3		
4			4			4			4		
5			5	2	1	5			5		
6			6			6			6		
7			7			7			7		
8	9	1	8			8			8		
9			9	22	1	9	125	1	9		
10			10			10			10		
11			11			11			11		
12			12			12			12	59	1
13			13			13			13		
14			14			14			14		
15			15			15			15		

In this system, if the CPU generates logical addresses I) **22135** and II) **41046** then map the corresponding physical addresses of these logical addresses. If the physical address is not valid then write “invalid”.

i) Address mapping (3-level paging)

Given:

- Logical address = 16 bits → split as **P1 (2 bits)**, **P2 (4 bits)**, **P3 (4 bits)**, **d (6 bits)** (page size 64 B ⇒ offset = 6 bits) Operating Systems-6-Memory Mana...
- Physical memory = **2 KB = 2048 B** → #frames = $2048/64 = 32$ → valid frame numbers **0–31** Operating Systems-6-Memory Mana...
- Physical address formula: **PA = (frame# × page size) + offset** Operating Systems-6-Memory Mana...

1) Logical address = 22135

Binary (16-bit): **0101011001110111**

So: **P1=01(1)**, **P2=0101(5)**, **P3=1001(9)**, **d=110111(55)**

Page-table walk (from given tables):

- 2nd Outer PMT: page 1 → frame **27**, **valid**
- Outer PMT (frame 27): page 5 → frame **2**, **valid**
- Inner PMT (frame 2): page 9 → frame **125**, **valid bit = 1**

But frame **125** is not in physical memory (only 0–31) ⇒ **invalid**

✓ Physical address for 22135: **INVALID**

2) Logical address = 41046

Binary (16-bit): 1010000001010110

So: P1=10(2), P2=1000(8), P3=0001(1), d=010110(22)

Page-table walk:

- 2nd Outer PMT: page 2 → frame 13, valid
- Outer PMT (frame 13): page 8 → frame 9, valid
- Inner PMT (frame 9): page 1 → frame 39, valid bit = 1

But frame 39 is not in physical memory (only 0–31) ⇒ invalid

✅ Physical address for 41046: INVALID

Spring25

a) Write the answer in the question paper.

Two users on a multi-user system launch the same text editor application at the same time. Instead of loading two separate copies of the editor's code into memory, the operating system loads only one copy which both processes use.

Identify which mechanism of paging is the operating system using to allow both processes to use the same code without duplicating it in memory?

The OS is using **shared pages** (shared code / reentrant read-only pages).

It keeps **one copy of the editor's read-only code** in memory and maps that same set of pages into **both processes' page tables**, so both can execute it without duplicating memory.

 Operating Systems-6-Memory Mana...

d) **Write the answer in the answer script.**

[10]

Shoumo has implemented an OS which has a page size of **8 KB** and each entry in the page table is sized **2 Bytes**. He has installed a new game and the new process has the logical address space of **64 bits**. Now in order to fit the pages of the process in the main memory, his OS will apply Two-Level Paging technique in outer page number bits until the outermost page table can be allocated in a frame of the main memory.

Illustrate the logical address space of the process including the necessary outer page bits, inner page bits and offset bits of every step with proper mathematical calculations during the paging mechanism of the system described above.

- Page size = 8 KB = 8192 B = $2^{13} \Rightarrow$ offset $d = 13$ bits Operating Systems-6-Memory Mana...
- Logical address space = 64 bits $\Rightarrow$ remaining bits for page number

$$64 - 13 = 51 \text{ bits (page number)}$$

- Page table entry size = 2 bytes
- Each page table must fit in **one frame (one page = 8KB)**

Step 1: How many entries fit in one page-table frame?

$$\text{entries per page table} = \frac{8192 \text{ B}}{2 \text{ B}} = 4096 = 2^{12}$$

So each page-table "page" needs 12 index bits (because it has 2^{12} entries). Operating Systems-6-Memory Mana...

Step 2: Split the 51-bit page number into 12-bit chunks (multi-level)

We keep splitting until the **outermost** table fits in one frame.

If each level uses 12 bits, then:

$$51 = 12 + 12 + 12 + 12 + 3$$

So we need **5 levels** of paging (4 full 12-bit levels + 1 top level with 3 bits).

- Level-1 index: **3 bits** $\Rightarrow 2^3 = 8$ entries
- Level-2 index: **12 bits** $\Rightarrow 4096$ entries
- Level-3 index: **12 bits** $\Rightarrow 4096$ entries
- Level-4 index: **12 bits** $\Rightarrow 4096$ entries
- Level-5 index: **12 bits** $\Rightarrow 4096$ entries
- Offset: **13 bits**



Step 3: Confirm the outermost page table fits in one frame

Outermost table entries = $2^3 = 8$

$$\text{size} = 8 \times 2 \text{ B} = 16 \text{ B} \ll 8192 \text{ B}$$

✓ Outermost page table fits in a single 8KB frame.

Also each inner-level table (12-bit index) has:

$$4096 \times 2 \text{ B} = 8192 \text{ B} = 8 \text{ KB}$$

✓ Exactly fits in one frame. [Operating Systems-6-Memory Mana...](#)

Final logical address format (what you should write)

$$\text{Logical Address (64 bits)} = p_1(3) p_2(12) p_3(12) p_4(12) p_5(12) d(13)$$

Where:

- d = offset within page (8KB)
- p_5 selects entry in the last-level page table (gives frame #)
- p_4, p_3, p_2 index intermediate page tables
- p_1 indexes the **outermost** page table that must fit in one frame

This is the hierarchical/multi-level paging idea: break the page number into multiple indices so page tables don't have to be huge and contiguous in memory. [Operating Systems-6-Memory Mana...](#)

Sources

- Page size = 8 KB = 8192 B = $2^{13} \Rightarrow$ offset $d = 13$ bits Operating Systems-6-Memory Mana...
- Logical address space = 64 bits $\Rightarrow$ remaining bits for page number

$$64 - 13 = 51 \text{ bits (page number)}$$

- Page table entry size = 2 bytes
- Each page table must fit in **one frame (one page = 8KB)**

Step 1: How many entries fit in one page-table frame?

$$\text{entries per page table} = \frac{8192 \text{ B}}{2 \text{ B}} = 4096 = 2^{12}$$

So **each page-table "page" needs 12 index bits** (because it has 2^{12} entries). Operating Systems-6-Memory Mana...

Step 2: Split the 51-bit page number into 12-bit chunks (multi-level)

We keep splitting until the **outermost** table fits in one frame.

If each level uses 12 bits, then:

$$51 = 12 + 12 + 12 + 12 + 3$$

So we need **5 levels** of paging (4 full 12-bit levels + 1 top level with 3 bits).

- Level-1 index: **3 bits** $\Rightarrow 2^3 = 8$ entries
- Level-2 index: **12 bits** $\Rightarrow 4096$ entries
- Level-3 index: **12 bits** $\Rightarrow 4096$ entries
- Level-4 index: **12 bits** $\Rightarrow 4096$ entries
- Level-5 index: **12 bits** $\Rightarrow 4096$ entries
- Offset: **13 bits**



Step 3: Confirm the outermost page table fits in one frame

Outermost table entries = $2^3 = 8$

$$\text{size} = 8 \times 2 \text{ B} = 16 \text{ B} \ll 8192 \text{ B}$$

✓ Outermost page table fits in a single 8KB frame.

Also each inner-level table (12-bit index) has:

$$4096 \times 2 \text{ B} = 8192 \text{ B} = 8 \text{ KB}$$

✓ Exactly fits in one frame.

Operating Systems-6-Memory Mana...

Final logical address format (what you should write)

Logical Address (64 bits) = $p_1(3) p_2(12) p_3(12) p_4(12) p_5(12) d(13)$

Where:

- d = offset within page (8KB)
- p_5 selects entry in the last-level page table (gives frame #)
- p_4, p_3, p_2 index intermediate page tables
- p_1 indexes the **outermost** page table that must fit in one frame

This is the hierarchical/multi-level paging idea: break the page number into multiple indices so page tables don't have to be huge and contiguous in memory.

Operating Systems-6-Memory Mana...

Sources



3

a) Write the answers in the question paper.

[3]

i) Kabbya was working on a file named log.txt. To perform a write operation on the file, the system must complete three steps:

- 1) Write the updated content to a data block
- 2) Update the corresponding bit in the data bitmap
- 3) Update the file's inode to reflect the new size and block pointers

While performing this write, the system unexpectedly lost power and crashed. After rebooting, Farid checked the file and noticed that the file size had not increased, and the changes he made were lost. Based on this post-crash state, **deduce** which of the following writes were successful and which ones failed:

- Data block write
- Data bitmap update
- Inode update

Answer:

Data block write	
Data bitmap update	
Inode update	

From the PDF's crash-consistency example: if **only the data block is written** but the **inode is not updated**, the system shows a **lost update** (file size doesn't increase and changes appear lost).

 Operating Systems-5-File System...

So, based on the post-crash observation (**file size did not increase and changes were lost**):

- **Data block write:**  **Successful** (it may have reached disk, but it is not referenced)

 Operating Systems-5-File System...

- **Data bitmap update:**  **Failed**  Operating Systems-5-File System...

- **Inode update:**  **Failed** (must fail, otherwise size would increase)  Operating Systems-5-File System...

d) **Write the answer in the answer script.**

[4]

An existing file named "a" needs to be read which is allocated in 4 data blocks.

- Path of the file: **"/f1/f2/a"**
- To read the file it was opened first by **open()** system call.
- After opening the file, **read()** system call was issued in the file to read the contents of the **2nd** and **3rd** data blocks only.

Illustrate the file access path timeline including proper order of all the operations according to the scenario described above.

File access-path timeline for reading only the 2nd and 3rd data blocks of `/f1/f2/a`

A) `open("/f1/f2/a", 0_RDONLY)` path traversal (disk reads)

1. Read root inode (start at `/`, root inode is used to locate root directory blocks).
Operating Systems-5-File System...
2. Read root directory data block(s) → find entry `f1` (directories store *(name, inode#)* pairs).
Operating Systems-5-File System...
3. Read inode of `f1`. Operating Systems-5-File System...
4. Read directory data block(s) of `f1` → find entry `f2`. Operating Systems-5-File System...
5. Read inode of `f2`. Operating Systems-5-File System...
6. Read directory data block(s) of `f2` → find entry `a`. Operating Systems-5-File System...
7. Read inode of file `a` (contains pointers to its 4 data blocks, size, protection, etc.).
Operating Systems-5-File System...
8. Check permissions + allocate file descriptor (this part is OS bookkeeping after traversal).
Operating Systems-5-File System...

****B)** `read()` to read the 2nd and 3rd data blocks only (disk reads + metadata update)

9. Using the inode's block pointers, read the 2nd data block of `a` from disk. Operating Systems-5-File System...
10. Update inode "last accessed time" (atime) and update the in-memory open-file offset/table.
Operating Systems-5-File System...
11. Using the inode's block pointers, read the 3rd data block of `a` from disk. Operating Systems-5-File System...
12. Update inode "last accessed time" (atime) and update the in-memory open-file offset/table.
Operating Systems-5-File System...

✅ That's the correct ordered timeline based on the PDF's "Access Paths: Reading a File From Disk" description (pathname traversal during `open()`, then `read()` uses the inode's pointers to fetch the needed blocks and updates access time/offset). Operating Systems-5-File System...

4. Write the answer in the answer script.

CO1 According to Ownership rights, **draw** the updated access matrix after one possible change we can make in (D2, F1) index.

Object Domain	F1	F2	F3
D1	Owner	Read	Read
D2	Read*	Owner	Owner
D3	Write	Read	Write

From the **Owner rights** rule: ownership allows adding or removing rights in an entry of the matrix.

Operating Systems-7-Protection...

Here, at index (D2, F1) the current right is Read*.

One possible owner-based change is to add "Write" for D2 on F1.

Updated access matrix (one valid example)

Domain \ Object	F1	F2	F3	
D1	Owner	Read	Read	
D2	Read*, Write	Owner	Owner	
D3	Write	Read	Write	

✓ Only (D2, F1) is changed, and it follows "Owner can add/remove rights."

Operating Systems-7-Protection...